AETHER

Autonomous Constellation Manager

Technical Report

National Space Hackathon 2026 – Indian Institute of Technology, Delhi

Team Name	UNIT313
Team Leader	Gaohar Imran
Team Member	Mohd Ali
Submission	March 2026

Orbital Debris Avoidance · Autonomous COLA · Constellation Station-Keeping · Real-Time Visualisation

This report details the numerical methods, spatial optimisation algorithms, and system architecture of AETHER, developed in response to the NSH 2026 problem statement. All physical constants, API specifications, and evaluation criteria are taken directly from the official problem statement.

Contents

1 Introduction **3**
1.1 Background and Motivation . . . 3
1.2 The AETHER System . . . 3

2 System Architecture **3**
2.1 Technology Stack . . . 3
2.2 Repository Structure (PS §9) . . . 4
2.3 Simulation Pipeline . . . 5

3 Orbital Propagation – Numerical Methods (PS §3) **5**
3.1 Reference Frame and State Vector (PS §3.1) . . . 5
3.2 Equations of Motion with J2 Perturbation (PS §3.2) . . . 5
3.3 4th-Order Runge-Kutta Integrator (PS §3.2) . . . 6
3.4 Vectorised Batch Propagation . . . 6
3.5 Debris Binary Blob Storage . . . 6

4 Spatial Optimisation Algorithms (PS §2) **6**
4.1 KD-Tree Conjunction Screening . . . 7
4.2 Time of Closest Approach (TCA) Solver . . . 7
4.3 Collision Probability (PS §3.3) . . . 7
4.3.1 Analytic 2-D Gaussian . . . 7
4.3.2 Monte Carlo Verification . . . 7

5 Propulsion and Fuel Model (PS §5.1) **7**
5.1 Tsiolkovsky Rocket Equation . . . 7
5.2 Command Latency (PS §5.4) . . . 8

6 Maneuvre Planning (PS §5.3) **8**
6.1 RTN Reference Frame . . . 8
6.2 24-Candidate Brute-Force Optimiser . . . 8
6.3 Station-Keeping Recovery . . . 8

7 Station-Keeping and Uptime (PS §5.2) **8**
7.1 Nominal Orbital Slot . . . 8
7.2 10 km Station-Keeping Box (PS §5.2) . . . 8
7.3 Satellite Status States . . . 9

8 Communication Constraints (PS §5.4) **9**
8.1 Ground Station Network (PS §5.5.1) . . . 9
8.2 Line-of-Sight Model . . . 9

9 REST API Specification (PS §4) **10**

10 Frontend: Orbital Insight Dashboard (PS §6) **10**
10.1 Performance Constraints (PS §6.1) . . . 10
10.2 Four Required Visualisation Modules (PS §6.2) . . . 10
10.3 Visualisation Snapshot Endpoint (PS §6.3) . . . 11

11 Deployment Requirements (PS §8) **11**
11.1 Multi-Stage Docker Build . . . 11

12 Evaluation Criteria Mapping (PS §7) **12**

13 Physical Constants Reference **13**

14 Conclusion **13**

1 Introduction

1.1 Background and Motivation

Low Earth Orbit (LEO) has rapidly transformed into a congested environment. Commercial mega-constellations have exponentially increased the number of active payloads, while millions of debris fragments—ranging from defunct rocket bodies to stray bolts—orbit at hypervelocity speeds exceeding 27,000 km/h. This severe congestion brings the orbital environment perilously close to the **Kessler Syndrome**: a cascading chain of collisions that could render entire orbital shells permanently unusable.

Current collision-avoidance practice is a heavily manual, human-in-the-loop process. Ground-based radar networks issue **Conjunction Data Messages (CDMs)** when a close approach is predicted, and Flight Dynamics Officers (FDOs) must manually evaluate each warning, calculate the necessary delta-v, and uplink thruster commands. This legacy approach suffers from three fundamental bottlenecks:

1. **Scalability:** Manual evaluation cannot scale to thousands of satellites receiving hundreds of CDM warnings daily.
2. **Communication blackouts:** When a satellite passes over deep oceans or poles, no ground station has line-of-sight. If a conjunction is predicted during a blackout, ground control is entirely helpless.
3. **Suboptimal fuel use:** Human operators struggle to globally optimise Δv expenditure across an entire fleet while simultaneously ensuring satellites return to their assigned orbital slots.

1.2 The AETHER System

AETHER (**A**utonomous **E**vasion and **T**rajectory **H**andling for **E**xo-atmospheric **R**esources) is a full-stack Autonomous Constellation Manager (ACM) developed for the National Space Hackathon 2026. It addresses all six NSH core objectives:

- High-frequency telemetry ingestion of ECI state vectors (PS §4.1)
- Predictive 24-hour conjunction assessment without $\mathcal{O}(N^2)$ complexity (PS §2)
- Autonomous collision avoidance (COLA) with RTN-frame manoeuvre planning (PS §5.3)
- Station-keeping and orbital recovery to the designated 10 km slot (PS §5.2)
- Tsiolkovsky-accurate fuel budgeting and EOL graveyard management (PS §5.1)
- Global multi-objective optimisation: maximise uptime while minimising Δv

2 System Architecture

2.1 Technology Stack

Layer	Technology
Backend API	FastAPI 0.115 (Python 3.10), uvicorn ASGI
State store	Redis 7 – ephemeral, no disk persistence
Physics engine	NumPy 1.26 + SciPy 1.15, custom RK4 + J2 propagator
Spatial index	SciPy `cKDTree` – $\mathcal{O}(N \log M)$
Frontend	React 18, Three.js r183, Recharts 2.5, Tailwind CSS 3
Container	Docker multi-stage, ubuntu:22.04 base (PS §8)

Table 1: Technology stack

2.2 Repository Structure (PS §9)

Listing 1: AETHER full-stack repository structure

```
aether/
  backend/
    main.py            # FastAPI -- REST API entrypoint

    physics/           # Orbital propagation + fuel model
      rk4.py
      j2.py
      propagator.py
      tsiolkovsky.py

    conjunction/       # Collision detection
      screening.py
      tca.py
      probability.py
      monte_carlo.py

    maneuver/          # COLA manoeuvre planning
      rtn.py
      planner.py
      executor.py

    optimizer/
      evasion.py

    spatial_algo/
      kdtree.py

    comms/
      los.py
      pass_predictor.py

    navigation/
      slot.py
      recovery.py

    storage/
      redis_client.py
      debris_blob.py

    utils/
      constants.py
      vector.py
      time.py

  frontend/
    src/
      components/
        Globe.jsx
        GroundTrack.jsx
        BullseyePlot.jsx
        FuelHeatmap.jsx
        Timeline.jsx

      services/
        api.js

      App.jsx
      main.jsx

    public/
```

```
    package.json

  Dockerfile
  entrypoint.sh
  README.md
```

2.3 Simulation Pipeline

Each `POST /api/simulate/step` call runs four sequential phases inside a dedicated worker thread (`asyncio.to_thread`), keeping the ASGI event loop free to serve `/api/status` throughout:

1. **Phase 1 – Propagation:** Vectorised batch RK4+J2 advances all objects simultaneously; nominal slots co-propagate (PS §3.2, PS §5.2).
2. **Phase 2 – Scheduled burns:** Queued commands whose `burnTime` has elapsed are executed, subject to LOS and cooldown gates (PS §4.2, PS §5.4).
3. **Phase 3 – Conjunction detection & COLA:** KD-tree screens all satellite–debris pairs; CRITICAL/WARNING events trigger autonomous RTN burns (PS §2, PS §5.3).
4. **Phase 4 – Station-keeping return:** Satellites flagged `needs_return` fire proportional RTN recovery burns (PS §5.2).

3 Orbital Propagation – Numerical Methods (PS §3)

3.1 Reference Frame and State Vector (PS §3.1)

All kinematic data uses the **Earth-Centred Inertial (ECI)** frame (J2000 epoch). Every object is defined by a 6-dimensional state vector:

$$S(t) = \begin{bmatrix} \boldsymbol{r}(t) \\ \boldsymbol{v}(t) \end{bmatrix} = \left[x,\, y,\, z,\, v_x,\, v_y,\, v_z\right]^T,$$

where position $\boldsymbol{r}$ is in km and velocity $\boldsymbol{v}$ in km/s.

3.2 Equations of Motion with J2 Perturbation (PS §3.2)

The propagation engine implements the J2-perturbed equations of motion exactly as specified in PS §3.2:

$$\frac{d^2\boldsymbol{r}}{dt^2} = -\frac{\mu}{|\boldsymbol{r}|^3}\,\boldsymbol{r} + \boldsymbol{a}_{J_2}, \tag{1}$$

where the J2 acceleration vector is:

$$\boldsymbol{a}_{J_2} = \frac{3\,J_2\,\mu\,R_E^2}{2\,|\boldsymbol{r}|^5} \begin{bmatrix} x\left(\dfrac{5z^2}{|\boldsymbol{r}|^2} - 1\right) \\ y\left(\dfrac{5z^2}{|\boldsymbol{r}|^2} - 1\right) \\ z\left(\dfrac{5z^2}{|\boldsymbol{r}|^2} - 3\right) \end{bmatrix}. \tag{2}$$

Physical constants from PS §3.2: $\mu = 398600.4418\,\mathrm{km}^3\,\mathrm{s}^{-2}$, $R_E = 6378.137\,\mathrm{km}$, $J_2 = 1.08263 \times 10^{-3}$

3.3 4th-Order Runge-Kutta Integrator (PS §3.2)

AETHER uses the classical RK4 scheme as required by PS §3.2:

$$\begin{aligned} \boldsymbol{k}_1 &= f(\boldsymbol{y}_n), & \boldsymbol{k}_2 &= f\left(\boldsymbol{y}_n + \tfrac{h}{2}\boldsymbol{k}_1\right), \\ \boldsymbol{k}_3 &= f\left(\boldsymbol{y}_n + \tfrac{h}{2}\boldsymbol{k}_2\right), & \boldsymbol{k}_4 &= f(\boldsymbol{y}_n + h\,\boldsymbol{k}_3), \\ \boldsymbol{y}_{n+1} &= \boldsymbol{y}_n + \frac{h}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4). \end{aligned} \tag{3}$$

Internal sub-steps of 60 s are used regardless of the tick size. For a 3,600 s fast-forward this gives 60 RK4 iterations; position error relative to a 5 s sub-step reference is $\approx 20\,\mathrm{m}$, well below the 100 m collision threshold of PS §3.3.

3.4 Vectorised Batch Propagation

All N objects are stacked into a single $(N \times 6)$ NumPy matrix and propagated in one vectorised call, replacing serial Python loops:

Listing 2: Module-level batch RK4 (closure overhead eliminated)

```python
def _batch_deriv(st: np.ndarray) -> np.ndarray:
    return np.hstack([st[:, 3:6], get_j2_acceleration(st)])

def _rk4_batch(states: np.ndarray, dt: float) -> np.ndarray:
    n = max(1, int(np.ceil(abs(dt) / 60.0)))    # 60-s sub-steps
    h = dt/n;  h2 = h*0.5;  h6 = h/6.0
    s = states.copy()
    for _ in range(n):
        k1 = _batch_deriv(s);            k2 = _batch_deriv(s + h2*k1)
        k3 = _batch_deriv(s + h2*k2);   k4 = _batch_deriv(s + h *k3)
        s += h6*(k1 + 2*k2 + 2*k3 + k4)
    return s
```

Method	Latency/3600s tick	Throughput
Serial Python loop (original)	∼9 s	∼110 obj/s
Vectorised NumPy batch (AETHER)	∼0.1 s	∼100,000 obj/s

Table 2: Propagation performance for 10,000 debris objects

3.5 Debris Binary Blob Storage

Storing 10,000 debris objects as individual Redis JSON keys requires 10,000 `SET` round-trips per step. AETHER serialises all debris states as a single packed NumPy binary array (`numpy.save`) stored under one Redis key:

- **Write:** `_write_debris_blob(states, ids)` – 1 Redis `SET`
- **Read:** `_read_debris_blob()` – 1 Redis `GET`
- Payload: $10{,}000 \times 6 \times 8$ bytes $= 480\,\mathrm{KB}$ vs $\sim 1.2\,\mathrm{MB}$ of JSON

4 Spatial Optimisation Algorithms (PS §2)

4.1 KD-Tree Conjunction Screening

Naïve all-pairs screening is $\mathcal{O}(NM)$ – with 50 satellites and 10,000 debris this is 500,000 distance evaluations per step. As required by PS §2, AETHER implements a SciPy `cKDTree` with a two-radius strategy to achieve $\mathcal{O}(N \log M)$:

1. **Outer radius** $r_{\text{outer}} = 50\,\text{km}$: candidate pairs for TCA analysis.
2. **Inner radius** $r_{\text{inner}} = 0.1\,\text{km}$ (100 m): hard collision flag as defined in PS §3.3: $|\boldsymbol{r}_{\text{sat}}(t) - \boldsymbol{r}_{\text{deb}}(t)| < 0.100\,\text{km}$.

4.2 Time of Closest Approach (TCA) Solver

For each candidate pair the TCA is found by a two-phase algorithm:

1. **Coarse scan:** Propagate both objects in 60 s steps over a 1-hour window; record the time t^* of minimum separation.
2. **Bisection refinement:** 30 iterations in $[t^* - 60, t^* + 60]$ to locate the TCA to $\approx 56\,\mu\text{s}$ precision ($60/2^{30}$).

4.3 Collision Probability (PS §3.3)

4.3.1 Analytic 2-D Gaussian

For combined position covariance σ and hard-body radius R:

$$P_c = 1 - \exp\left(-\frac{R^2}{2\sigma^2}\right). \tag{4}$$

4.3.2 Monte Carlo Verification

Events with $P_c > 10^{-3}$ are passed through a 5,000-sample Monte Carlo: each sample perturbs both state vectors with Gaussian noise, propagates to the TCA, and counts intersections. This acts as a second gate before any avoidance burn is committed.

5 Propulsion and Fuel Model (PS §5.1)

5.1 Tsiolkovsky Rocket Equation

The propellant mass consumed per burn is computed using the Tsiolkovsky equation *exactly as given in* PS §5.1, using m_{current} (dynamic, decreases as fuel depletes):

$$\Delta m = m_{\text{current}} \left(1 - e^{-|\Delta \boldsymbol{v}| \,/\, (I_{sp} \cdot g_0)}\right), \tag{5}$$

where $m_{\text{current}} = m_{\text{dry}} + m_{\text{fuel,remaining}}$ is tracked dynamically. As fuel depletes, the satellite becomes lighter and subsequent burns become marginally more efficient, as specified in PS §5.1.

Spacecraft constants from PS §5.1 – identical for all satellites:

- Dry mass $m_{\text{dry}} = 500.0\,\text{kg}$
- Initial propellant $m_{\text{fuel,0}} = 50.0\,\text{kg}$ (wet mass = 550.0 kg)
- Specific impulse $I_{sp} = 300.0\,\text{s}$
- Standard gravity $g_0 = 9.80665\,\text{m}\,\text{s}^{-2}$
- Maximum $|\Delta \boldsymbol{v}|$ per burn: $15.0\,\text{m}\,\text{s}^{-1}$
- Thermal cooldown between burns: 600 s
- EOL fuel threshold: 5% of $m_{\text{fuel,0}} \rightarrow$ graveyard orbit

5.2 Command Latency (PS §5.4)

All burn commands carry a hardcoded 10-second signal delay. The satellite state is propagated by $\Delta t = 10\,\mathrm{s}$ using RK4 before applying the impulsive $\Delta\boldsymbol{v}$, ensuring the manoeuvre is applied at the correct future state. A burn cannot be scheduled earlier than `current_sim_time + 10 s`.

6 Maneuvre Planning (PS §5.3)

6.1 RTN Reference Frame

All $\Delta\boldsymbol{v}$ vectors are planned in the Radial-Transverse-Normal (RTN) frame as required by PS §5.3:

$$\hat{\boldsymbol{u}}_R = \frac{\boldsymbol{r}}{|\boldsymbol{r}|}, \qquad \hat{\boldsymbol{u}}_N = \frac{\boldsymbol{r}\times\boldsymbol{v}}{|\boldsymbol{r}\times\boldsymbol{v}|}, \qquad \hat{\boldsymbol{u}}_T = \hat{\boldsymbol{u}}_N \times \hat{\boldsymbol{u}}_R. \tag{6}$$

The $\Delta\boldsymbol{v}$ RTN vector is then rotated back to ECI before submission to the manoeuvre API.

6.2 24-Candidate Brute-Force Optimiser

AETHER evaluates 24 candidate $\Delta\boldsymbol{v}$ directions on the unit sphere and selects the direction maximising post-manoeuvre miss distance while minimising $|\Delta\boldsymbol{v}|$:

$$\mathrm{score}(c) = 10 \cdot d_{\mathrm{post}}(c) - |\Delta\boldsymbol{v}(c)| \quad \mathrm{km/s}, \tag{7}$$

with the burn magnitude capped at $\Delta v_{\max} = 0.015\,\mathrm{km/s}$ per PS §5.1. The prograde/retrograde direction (PS §5.3 "Phasing Manoeuvre") is the most fuel-efficient choice and is always included as a candidate.

6.3 Station-Keeping Recovery

Per PS §5.2, every evasion manoeuvre must be paired with a recovery trajectory. AETHER implements proportional RTN correction with gain $K = 10^{-3}\,\mathrm{km\,s^{-1}\,km^{-1}}$ (1 m/s per km of error), capped at 10 m/s, subject to the 600 s cooldown and LOS gates.

7 Station-Keeping and Uptime (PS §5.2)

7.1 Nominal Orbital Slot

Each satellite's nominal slot is its 6-DOF state at first ingest. On every simulation step, the nominal slot is co-propagated through the same RK4+J2 integrator as the satellite. This ensures the reference point moves along the satellite's *ideal unperturbed orbit* (PS §5.2), so a satellite on a perfect Keplerian trajectory registers zero drift. Only physical perturbations or avoidance burns create measurable displacement.

7.2 10 km Station-Keeping Box (PS §5.2)

A satellite is "Nominal" when:

$$|\boldsymbol{r}_{\mathrm{sat}}(t) - \boldsymbol{r}_{\mathrm{nominal}}(t)| \le 10\,\mathrm{km}. \tag{8}$$

The system tracks two cumulative timers per satellite:

$$\mathrm{Uptime}_i = \frac{t^{(i)}_{\mathrm{inside}}}{t^{(i)}_{\mathrm{inside}} + t^{(i)}_{\mathrm{outside}}} \times 100\%. \tag{9}$$

7.3 Satellite Status States

The status field in the API reflects the satellite's operational state:

- `NOMINAL` – within 10 km of assigned slot (PS §5.2)
- `WARNING` – outside 10 km box, recovery burn pending
- `GRAVEYARD` – fuel $\leq 5\%$ threshold, moved to safe disposal orbit (PS §2)

8 Communication Constraints (PS §5.4)

8.1 Ground Station Network (PS §5.5.1)

The system uses the six ground stations provided in `ground_stations.csv`:

ID	Station	Lat	Lon	Elev. m	Min El.
GS-001	ISTRAC Bengaluru	13.0333N	77.5167E	820	5.0°
GS-002	Svalbard Sat. Station	78.2297N	15.4077E	400	5.0°
GS-003	Goldstone Tracking	35.4266N	116.8900W	1000	10.0°
GS-004	Punta Arenas	53.1500S	70.9167W	30	5.0°
GS-005	IIT Delhi Ground Node	28.5450N	77.1926E	225	15.0°
GS-006	McMurdo Station	77.8463S	166.6682E	10	5.0°

Table 3: Ground station network (from `ground_stations.csv`, PS §5.5.1)

8.2 Line-of-Sight Model

For each ground station the LOS check performs:

1. Earth rotation correction: station ECI position computed using $\omega_E = 7.292115 \times 10^{-5}\,\mathrm{rad\,s^{-1}}$.
2. Parametric ray-sphere intersection test between station and satellite to detect Earth occlusion.
3. Elevation angle verification against per-station minimum mask.

Avoidance and station-keeping burns are **rejected** if no ground station has LOS at the scheduled uplink time (PS §5.4).

9 REST API Specification (PS §4)

Method	Endpoint	Spec.
POST	`/api/telemetry`	Ingest ECI state vectors (PS §4.1)
POST	`/api/maneuver/schedule`	Queue burn sequence + validation (PS §4.2)
POST	`/api/simulate/step`	Advance simulation by `step_seconds` (PS §4.3)
GET	`/api/visualization/snapshot`	Compact frontend snapshot (PS §6.3)
GET	`/api/conjunction/forecast`	24-hour CDM prediction (PS §2)
GET	`/api/system/metrics`	Fleet stats for PS §7 evaluation
GET	`/api/status`	Docker health check (PS §8)
GET	`/api/alerts/history`	CDM event log with all required fields
POST	`/api/reset`	Clear all simulation state

Table 4: REST API endpoints

PS §4.1 Telemetry Response

```
{"status": "ACK", "processed_count": 58, "active_cdm_warnings": 3}
```

PS §4.2 Manoeuvre Schedule Response

```
{"status": "SCHEDULED",
 "validation": {"ground_station_los": true, "sufficient_fuel": true,
                "projected_mass_remaining_kg": 548.12}}
```

PS §4.3 Simulation Step Response

```
{"status": "STEP_COMPLETE", "new_timestamp": "2026-03-12T09:00:00Z",
 "collisions_detected": 0, "maneuvers_executed": 2}
```

10 Frontend: Orbital Insight Dashboard (PS §6)

The dashboard is a React 18 SPA served at `GET /` from the same port-8000 FastAPI process.

10.1 Performance Constraints (PS §6.1)

The visualiser renders **50+ active satellites and 10,000+ debris objects in real-time at 60 FPS** using the Canvas API and WebGL (Three.js r183), as required by PS §6.1. DOM manipulation is avoided entirely; all objects are drawn via batch `THREE.Points` geometry (single GPU draw call per frame).

10.2 Four Required Visualisation Modules (PS §6.2)

1. **Ground Track Map (Mercator Projection):** Dynamic 2D world map with real-time satellite markers, the last 90 minutes of historical trail, a dashed predicted trajectory for the next 90 minutes, and a dynamic Terminator Line shadow overlay (PS §6.2).
2. **Conjunction Bullseye Plot (Polar Chart):** The selected satellite is fixed at the origin; radial distance encodes Time to Closest Approach (TCA); angle encodes the relative approach vector; debris markers are colour-coded by risk: Green = Safe (> 5 km), Yellow = Warning (< 5 km), Red = Critical (< 1 km) (PS §6.2).

3. **Telemetry & Resource Heatmaps:** Visual fuel gauge showing m_{fuel} for every satellite, and a Δv cost analysis graph plotting "Fuel Consumed" vs "Collisions Avoided" (PS §6.2).
4. **Manoeuvre Timeline (Gantt Scheduler):** Chronological schedule showing burn start/end blocks, the mandatory 600 s thruster cooldown periods, and LOS blackout zone overlaps (PS §6.2).

10.3 Visualisation Snapshot Endpoint (PS §6.3)

The compact snapshot format matches PS §6.3 exactly:

Listing 3: PS §6.3 snapshot response format

```
{
  "timestamp": "2026-03-12T08:00:00.000Z",
  "satellites": [
    {"id":"SAT-Alpha-04","lat":28.545,"lon":77.192,
     "fuel_kg":48.5,"status":"NOMINAL"}
  ],
  "debris_cloud": [
    ["DEB-99421", 12.42, -45.21, 400.5],    # [ID, lat, lon, alt_km]
    ["DEB-99422", 12.55, -45.10, 401.2]
  ]
}
```

The status field reflects the PS-defined states: `NOMINAL` (in box), `WARNING` (outside box), or `GRAVEYARD`. ECI-to-geodetic conversion for all 10,000 debris objects runs as a single vectorised NumPy operation in < 2 ms and executes in a thread pool to prevent event-loop blocking.

11 Deployment Requirements (PS §8)

PS §8 Hard Requirements – all satisfied (disqualification if violated):
✓ `Dockerfile` at repository root
✓ Base image `ubuntu:22.04`
✓ `EXPOSE 8000`; uvicorn binds to `0.0.0.0:8000`
✓ Single `docker run -p 8000:8000 aether` starts the full system

11.1 Multi-Stage Docker Build

Listing 4: Dockerfile structure (abbreviated)

```
# Stage 1: compile React to static files (Node.js discarded afterwards)
FROM node:18-slim AS node-builder
WORKDIR /build
COPY frontend/package*.json ./
RUN npm ci --prefer-offline --no-audit
COPY frontend/ .
ENV NODE_ENV=production CI=false
RUN npm run build

# Stage 2: PS \S{}8 mandatory base image
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.10 python3-pip redis-server curl && \
    apt-get clean && rm -rf /var/lib/apt/lists/*
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY backend/ ./backend/
COPY --from=node-builder /build/build ./frontend/build
```

```
EXPOSE 8000
CMD ["/app/entrypoint.sh"]
```

The multi-stage build discards the Node.js runtime after compilation, reducing the final image by $\approx 400\,\text{MB}$. Redis starts inside the same container via `entrypoint.sh` – no external service dependencies.

12 Evaluation Criteria Mapping (PS §7)

Criterion	Metric	AETHER Implementation
Safety (25%)	Conjunctions avoided	KD-tree $\mathcal{O}(N \log M)$ screening + TCA bisection + Monte Carlo gate + 24-candidate RTN burn optimizer; autonomous avoidance fires within the same simulation tick
Fuel Efficiency (20%)	Total fleet Δv	Optimiser minimises $\lvert\Delta v\rvert$ subject to miss-distance constraint; Tsiolkovsky with m_{current} (dynamic) per PS §5.1
Uptime (15%)	Time within 10 km box	Nominal slot co-propagated each step (PS §5.2); uptime $= t_{\text{inside}}/(t_{\text{inside}} + t_{\text{outside}})$; status: NOMINAL/WARNING/GRAVEYARD
Speed (15%)	Step latency	Vectorised batch RK4 ($(N \times 6)$ matrix); debris binary blob (1 Redis op); `asyncio.to_thread` isolation; benchmark: 1.52 s per 3,600 s tick at 50 sats + 10k debris
UI/UX (15%)	Dashboard quality	WebGL 60 FPS globe; all four PS §6.2 modules; 71 ms snapshot latency; real-time polling with auto-tab switching
Code (10%)	Modularity, logging	7 physics/algorithm packages; PS section references in every file; CDM history logged with all required fields

Table 5: NSH evaluation criteria and AETHER implementations

13 Physical Constants Reference

Symbol	Value	Unit	Source
μ	398600.4418	km^3s^{-2}	PS §3.2
R_E	6378.137	km	PS §3.2
J_2	1.08263×10^{-3}	–	PS §3.2
ω_E	7.292115×10^{-5}	rad s^{-1}	IERS
I_{sp}	300.0	s	PS §5.1
g_0	9.80665	m s^{-2}	PS §5.1
m_{dry}	500.0	kg	PS §5.1
$m_{\text{fuel},0}$	50.0	kg	PS §5.1
m_{wet}	550.0	kg	PS §5.1
$\Delta v_{\max}$	0.015	km s^{-1}	PS §5.1
t_{cooldown}	600.0	s	PS §5.1
t_{latency}	10.0	s	PS §5.4
r_{box}	10.0	km	PS §5.2
D_{crit}	0.100	km	PS §3.3
f_{EOL}	5.0	%	PS §2

Table 6: Complete mission constants (all sourced directly from the problem statement)

14 Conclusion

AETHER demonstrates that a physically rigorous, real-time autonomous constellation manager can be built within a compact Python/React stack while strictly adhering to every constraint in the NSH 2026 problem statement. The four engineering decisions that enable full-scale performance are:

1. **Vectorised numerics:** Replacing serial Python loops with NumPy matrix operations achieves a $> 100\times$ speedup in the critical computation path, enabling 1.52 s step latency at 10,000+ objects.
2. **Binary blob storage:** Serialising all debris states into one packed NumPy array reduces Redis I/O from 20,000 round-trips to 2 per step.
3. **Async thread isolation:** Running `simulate_step` and the visualisation snapshot on worker threads via `asyncio.to_thread` keeps the ASGI event loop responsive, eliminating false "backend offline" readings on the dashboard.
4. **Co-propagated nominal slots:** Advancing the station-keeping reference alongside the satellite on the same RK4+J2 orbit eliminates false out-of-box readings caused by ordinary orbital motion, giving physically meaningful uptime measurements.

All response formats, physical constants, API fields, and evaluation behaviours are derived directly from the NSH 2026 problem statement.

Team UNIT313

Gaohar Imran (Team Leader) · Mohd Ali

National Space Hackathon 2026 · IIT Delhi · March 2026